Smart Traffic Monitoring and Vehicle Classification System

Project Report

26MAI007:Subham Sharma 26MAI015:Rudradatt Purohit

1. Aim

The aim of this project is to build a system that can look at road images and traffic videos, recognise the type of each vehicle, follow it from frame to frame and count the vehicles that cross a chosen line. From these counts we also want a simple idea of how congested the road is, so that traffic does not have to be studied by watching hours of footage by hand.

2. Objective

1. To classify vehicles into seven classes: auto rickshaw, bicycle, bus, car, motorcycle, scooter and truck. These classes were chosen because Indian roads carry a mix of vehicles that many standard datasets do not cover well, especially auto rickshaws and scooters.
2. To train our own CNN from scratch and compare it with two ready-made models, ResNet-18 and MobileNetV3-Small, on accuracy, speed and model size.
3. To detect and track vehicles in video and count each vehicle only once when it crosses a virtual line, along with its direction of travel.
4. To estimate the congestion level of the road as Low, Medium or High.
5. To store the results in a database and show them in a simple web dashboard.

3. Methodology

3.1 Dataset

The dataset has 7,035 images in seven classes. The images were arranged in one folder per class and split into train, validation and test sets in roughly the ratio 70:15:15, which gave 4,852 training images, 1,081 validation images and 1,102 test images. The split is stratified, so every class keeps about the same share in each set, and a fixed random seed is used so the same split can be repeated. Frames taken from the same video look almost identical, and if such frames are spread over both the training and the test set the accuracy will look better than it really is. The split script gives a warning about this problem.

Figure 1 shows how many images each class has. The data is clearly not balanced. Car is the largest class with about 3,000 images, followed by motorcycle about 1,800 and scooter about 900. Bus and bicycle have a little under 600 images each. Auto rickshaw and truck are very small classes, with only 24 and 15 images in the test set. To reduce the effect of this

imbalance, class weights were used in the loss function, so that mistakes on small classes count more during training. Training images were also augmented with random flips, small rotations and colour changes.

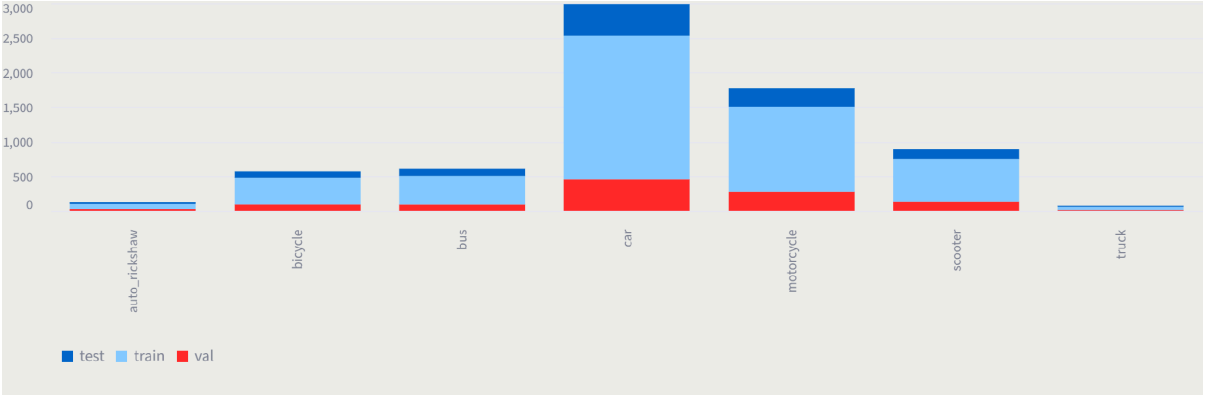


Figure 1: Number of images per class in the train, validation and test sets

3.2 Classification models and training

Three models were trained and compared. The first is a custom CNN with four convolution blocks. Its convolution, batch normalisation, pooling and backpropagation are written by hand in NumPy instead of using ready made PyTorch layers, and it is connected to PyTorch only so that a standard optimizer can update its weights. The second is ResNet-18 and the third is MobileNetV3-Small. Both of these are pretrained models whose last layer was replaced to give seven outputs.

All models were trained with the AdamW optimizer and a cosine learning rate schedule. Training can stop early if the validation loss does not improve for five epochs, and the best checkpoint of each model is saved. After training, each model was tested on the held out test set.

3.3 Detection and tracking

For video, a YOLO detector finds the vehicles in every frame and the ByteTrack tracker gives each vehicle an ID that stays the same as it moves. A pretrained YOLO model uses the COCO labels, which have no class for auto rickshaw or scooter, and which treat scooters as motorcycles. To handle this, the cropped image of each detected vehicle is passed to our own seven-class classifier, which gives the final label. Since the label can change from frame to frame, the final class of a vehicle is decided by majority vote over the frames in which it was seen.

3.4 Counting and congestion estimate

A virtual line is drawn across the road in the video frame. A vehicle is counted when the path of its centre point crosses this line. The crossing is checked with a counter clockwise orientation test, which needs no division and therefore does not fail when the line or the path is vertical. The ID of every counted vehicle is stored so that one vehicle is never counted

twice, and the direction coming or going is found from the cross product of the line and the movement of the vehicle.

Congestion is estimated from the number of vehicles in the frame and the share of the road area covered by their bounding boxes, and is put into the Low, Medium or High group. Speed in km/h was deliberately not reported, because it needs a calibrated camera and known distances on the road, which we did not have.

3.5 Database, dashboard and testing

Training runs, model comparisons and traffic sessions are saved in an SQLite database. A Streamlit web app with nine pages lets the user check the dataset, train models, compare them, classify a single image, upload a video for counting, view past sessions and download the results as CSV files. The code was checked with 26 automated unit tests written in pytest, covering the models, dataset handling, evaluation metrics and the line-crossing logic.

4. Result

4.1 Training of the custom CNN

The custom CNN was trained for 20 epochs. Figure 2 shows the loss and accuracy curves. The training loss went down steadily from about 1.44 in the first epoch to about 0.62 at the end, and the training accuracy rose from about 58% to about 82%. The validation curves were unsteady in the first ten epochs, with sharp ups and downs, and became much smoother after about epoch 12. The validation loss ended near 0.45 and the validation accuracy at about 83%. The validation loss stays below the training loss in the plot. This is probably because augmentation and dropout are active only during training. The two accuracy curves end close to each other, so there is no sign of overfitting.

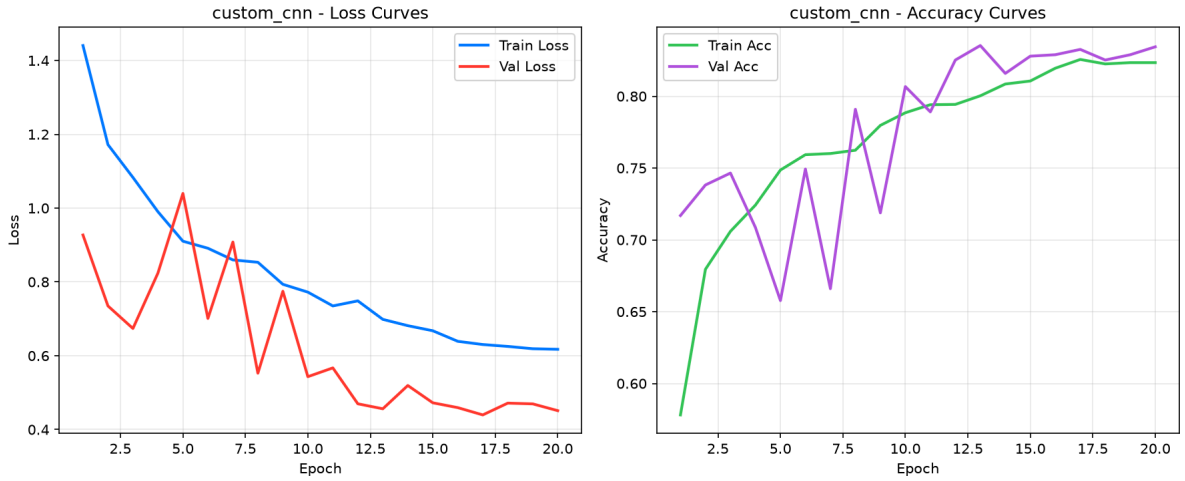


Figure 2: Loss and accuracy curves of the custom CNN over 20 epochs

4.2 Comparison of the three models

Table 1 gives the accuracy figures of the three models on the test set of 1,102 images and Table 2 gives their size and speed. Figure 3 shows the same comparison as charts.

Table 1: Test set performance

Model	Accuracy (%)	Macro precision (%)	Macro recall (%)	Macro F1 (%)
Custom CNN	84.21	76.36	77.94	76.93
ResNet-18	78.13	71.02	73.30	70.91
MobileNetV3-Small	71.51	65.25	67.78	62.85

Table 2: Size and speed

Model	Parameters	Checkpoint size (MB)	Latency (ms/image)	Speed (FPS)
Custom CNN	98,823	1.16	241.36	4.14
ResNet-18	11,180,103	128.07	10.09	99.07
MobileNetV3-Small	1,525,031	17.70	11.53	86.73

The custom CNN gave the best accuracy 84.2% and the best macro F1 76.9%, even though it is by far the smallest model with under 0.1 million parameters. ResNet-18 came second with 78.1% accuracy and MobileNetV3-Small was last with 71.5%. For all three models the macro F1 is lower than the accuracy. This is expected with our data, because macro F1 gives every class the same weight and the small or similar-looking classes are harder for the models than the large ones.

Speed shows a very different order. ResNet-18 and MobileNetV3-Small take about 10 to 12 ms per image, which is around 87 to 99 frames per second, while the custom CNN takes about 241 ms per image, or roughly 4 frames per second. This is because the custom CNN runs its operations in NumPy on the CPU and cannot use the GPU, whereas the other two run on the GPU through PyTorch. The checkpoint sizes in Table 2 also include the optimizer state, which is why they are larger than the weights alone.

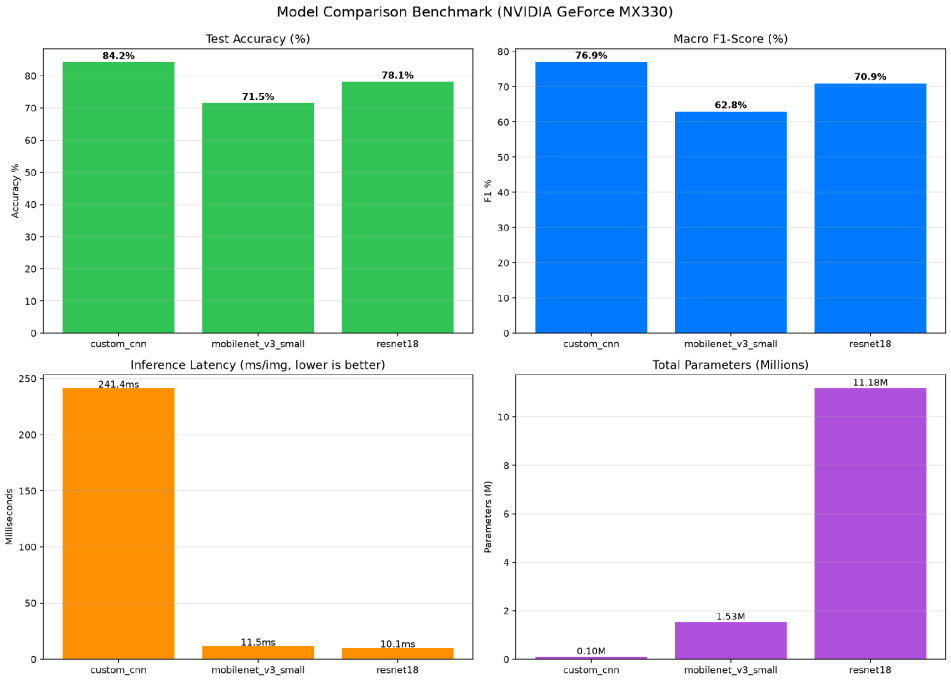


Figure 3: Model comparison on test accuracy, macro F1, latency and number of parameters

4.3 Class wise result of the custom CNN

Figure 4 is the confusion matrix of the custom CNN on the test set, and Table 3 summarises it class by class. Of the 1,102 test images, 928 were classified correctly. Car is recognised almost perfectly 450 of 453. Bus 87%, auto rickshaw 83% and bicycle 80% are also recognised well, and motorcycle is at 77%.

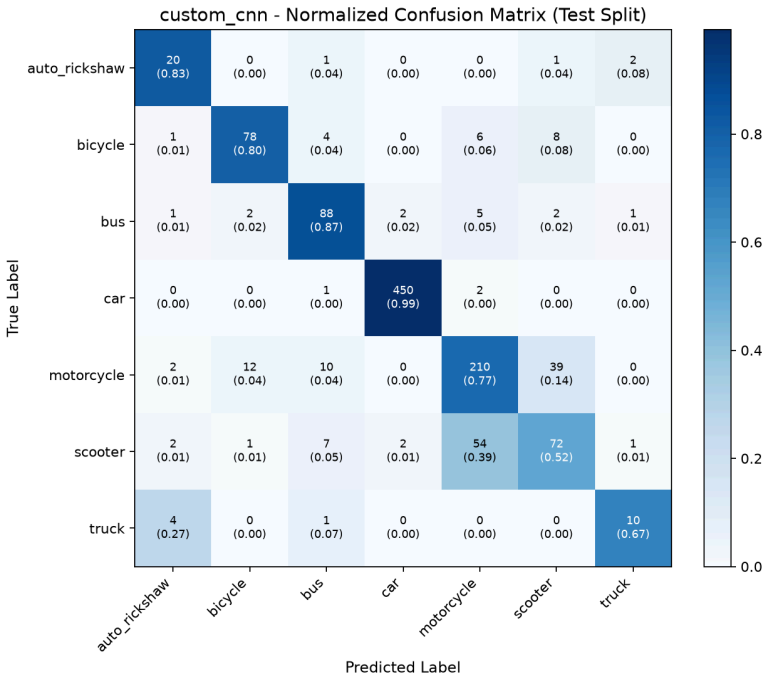


Figure 4: Normalised confusion matrix of the custom CNN on the test set

Table 3: Class wise result of the custom CNN on the test set

Class	Test images	Correct	Recall (%)
auto_rickshaw	24	20	83
bicycle	97	78	80
bus	101	88	87
car	453	450	99
motorcycle	273	210	77
scooter	139	72	52
truck	15	10	67

The weakest class is scooter, with a recall of only 52%. Of the 139 scooter images, 54 were predicted as motorcycle, and in the other direction 39 of the 273 motorcycles were predicted as scooter. These two vehicles look alike from many angles, so this is the largest source of error in the whole model. Among the other mistakes, 4 of the 15 trucks were predicted as auto rickshaw. Truck and auto rickshaw have very few test images (15 and 24), so each single image changes their percentage by 4 to 7 points, and their recall values should be treated as rough.

4.4 Single image classification in the dashboard

Figure 5 shows the Classify image page of the dashboard. A cropped photo of a scooter carrying three people was uploaded, and the custom CNN predicted Scooter with a confidence of 61.87%. The probability chart on the right shows that truck got the second highest probability, at about 19%, which means the model was not very sure. The page showed an inference time of about 6.3 seconds for this single image, which is much more than the 241 ms measured in the benchmark.

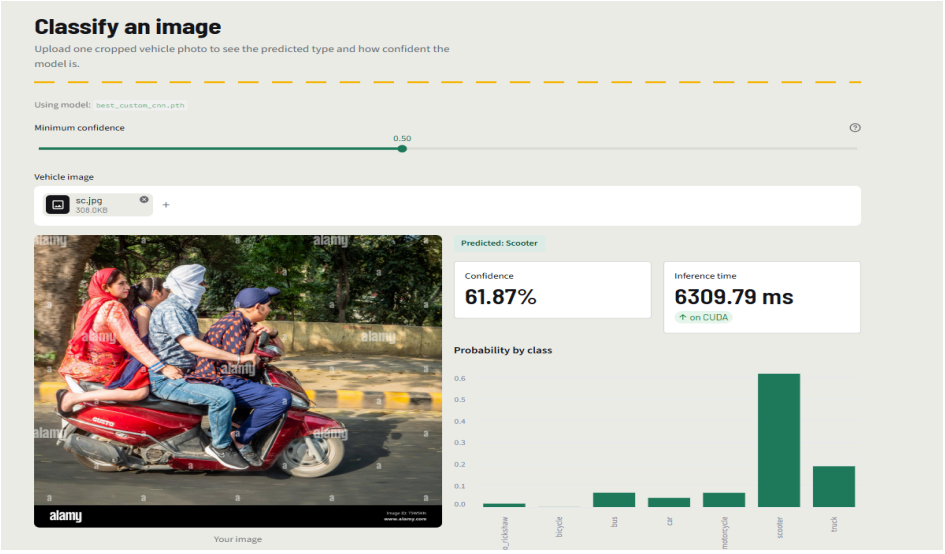


Figure 5: Classify image page of the dashboard

4.5 Video tracking result

Figure 6 is a frame from the traffic video processed by the system. The road is a busy multi lane road with cars, trucks, two wheelers and auto rickshaws moving together. Each tracked vehicle has a coloured box, an ID number, a predicted class with its confidence, and a small trail that shows where it has moved. The tracker kept IDs on many vehicles at the same time, which is what the counting step depends on.

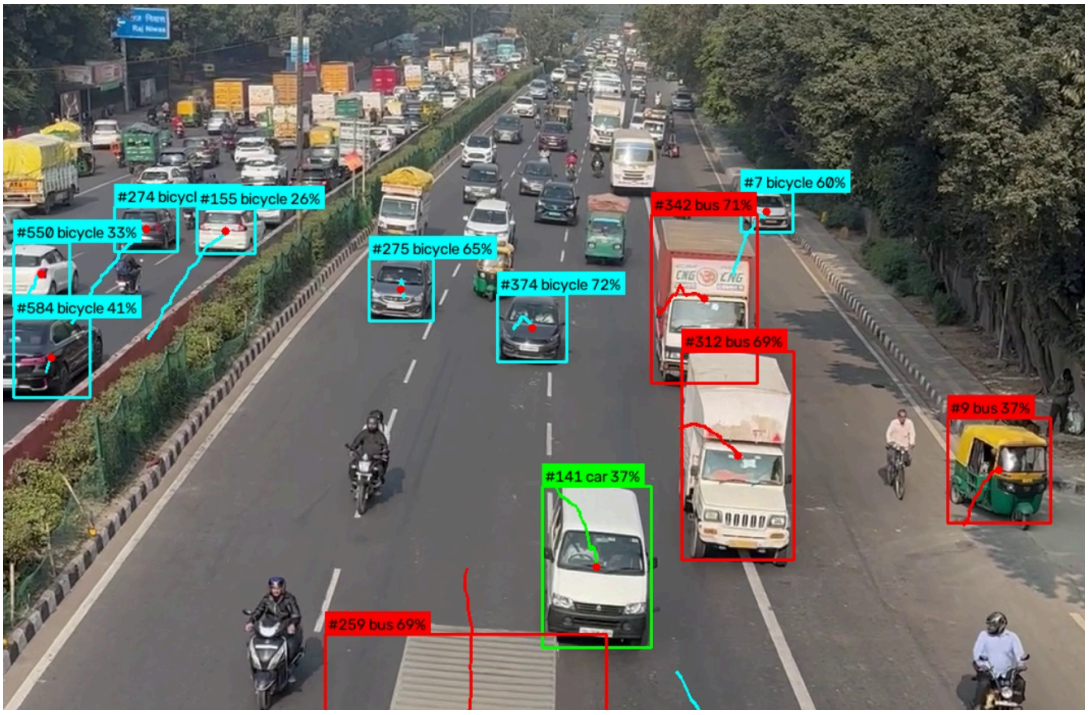


Figure 6: Detection and tracking on a traffic video frame

The frame also shows that the class labels are not always correct. Several cars were labelled as bicycle, with confidence between about 26% and 72%. The two goods trucks on the right side were labelled as bus, and the auto rickshaw at the right edge was also labelled as bus with only 37% confidence. A van was labelled car with 37% confidence. A few two-wheelers visible in the frame did not have a box at the moment this frame was captured. On the clean test images the classifier did much better, for example 20 of 24 auto rickshaws and 10 of 15 trucks were correct. So the video errors are probably not only because of the small number of such images, but also because vehicles seen from a bridge at a distance are small and look different from the training images.

5. Conclusion

In this project we built a working pipeline that classifies vehicles into seven classes, tracks them in video, counts line crossings without double counting, and shows everything in a web dashboard backed by a database. In the model comparison, the custom CNN gave the best accuracy 84.2% and macro F1 76.9% and is also the smallest model, so it is a good choice when accuracy and memory matter more than speed. Its training was stable and showed no

sign of overfitting. ResNet-18 was the fastest at about 99 FPS with 78.1% accuracy, and is the more practical choice when frames must be processed in real time. MobileNetV3-Small gave the lowest accuracy of the three in our experiments.

The work also has clear limits. The dataset is heavily unbalanced, and the small classes, truck and auto rickshaw, have only 15 and 24 test images, so the results for them are not very reliable. Scooter and motorcycle are often confused with each other, and scooter recall is only 52%. In the video, trucks and auto rickshaws were often labelled wrongly and many predictions had low confidence. Congestion is only an estimate from the image, and vehicle speed was not measured because the camera was not calibrated. The test video results were checked by looking at output frames and not against a hand-counted total.

For future work, the first step should be to collect more images of scooters, auto rickshaws and trucks, and to train a YOLO detector directly on the seven classes with boxes drawn on real road footage, so that the extra classification step is not needed. Moving the custom CNN to run on the GPU would also remove its speed problem. Camera calibration would allow real speed and density values, and testing on more roads, times of day and weather would show how well the system works in general.